

5.8 Input File (Upload de fichiers)

Listing 21 – Upload image avec prévisualisation

```
import { useState } from 'react';

function ImageUploadForm() {
  const [image, setImage] = useState(null);
  const [preview, setPreview] = useState('');

  const handleImageChange = (event) => {
    const file = event.target.files[0];

    if (file) {
      setImage(file);

      // Créer URL de prévisualisation
      const reader = new FileReader();
      reader.onloadend = () => {
        setPreview(reader.result);
      };
      reader.readAsDataURL(file);
    }
  };

  return (
    <div>
      <input
        type="file"
        accept="image/*"
        onChange={handleImageChange}
      />

      {preview && (
        <div>
          <p>Prévisualisation :</p>
          <img
            src={preview}
            alt="Preview"
            style={{ width: '200px', height: 'auto' }}
          />
          <p>Nom : {image.name}</p>
          <p>Taille : {(image.size / 1024).toFixed(2)} KB</p>
        </div>
      )}
    </div>
  );
}
```

★ Astuce

Input file particularité :

Les input file ne peuvent pas être "contrôlés" au sens strict (pas de `value`). On utilise `onChange` pour récupérer le fichier via `event.target.files[0]`.

5.9 Formulaire Complet Multi-Champs

Listing 22 – Formulaire avec tous types d'inputs

```
function CompleteForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: '',
    gender: '',
    category: '',
    newsletter: false,
    preferences: {
      cuisine: false,
      mode: false
    }
  });

  const handleChange = (event) => {
    const { name, value, type, checked } = event.target;

    if (type === 'checkbox') {
      setFormData({
        ...formData,
        [name]: checked
      });
    } else {
      setFormData({
        ...formData,
        [name]: value
      });
    }
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    console.log('Données formulaire:', formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="name"
        value={formData.name}
        onChange={handleChange}
        placeholder="Nom complet"
      />

      <input
        name="email"
        type="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Email"
      />

      <textarea
        name="message"
        value={formData.message}
      />
    </form>
  );
}
```

```

        onChange={handleChange}
        placeholder="Message"
      />

      <select name="category" value={formData.category} onChange={
        handleChange}>
        <option value="">Catégorie</option>
        <option value="cuisine">Cuisine</option>
        <option value="mode">Mode</option>
      </select>

      <label>
        <input
          type="checkbox"
          name="newsletter"
          checked={formData.newsletter}
          onChange={handleChange}
        />
        Newsletter
      </label>

      <button type="submit">Envoyer</button>
    </form>
  );
}

```

5.10 Validation de Formulaire

Listing 23 – Formulaire avec validation

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  const validateForm = () => {
    const newErrors = {};

    // Validation email
    if (!email) {
      newErrors.email = 'Email requis';
    } else if (!email.includes('@')) {
      newErrors.email = 'Email invalide';
    }

    // Validation mot de passe
    if (!password) {
      newErrors.password = 'Mot de passe requis';
    } else if (password.length < 6) {
      newErrors.password = 'Minimum 6 caractères';
    }

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = (event) => {
    event.preventDefault();

    if (validateForm()) {
      console.log('Formulaire valide !', { email, password });
      // Envoyer au serveur
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Email"
        />
        {errors.email && <p style={{ color: 'red' }}>{errors.email}</p>}
      </div>

      <div>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Mot de passe"
        />
      </div>
    </form>
  );
}
```

```
        {errors.password && <p style={{ color: 'red' }}>{errors.password  
          }</p>}  
      </div>  
  
      <button type="submit">Se connecter</button>  
    </form>  
  );  
}
```

★ Astuce

Validation en temps réel vs à la soumission :

En temps réel : Valider dans `onChange` (meilleure UX)

À la soumission : Valider dans `onSubmit` (plus simple)

PAUSE COURS - Exercice Pratique Complet

Exercice : Créer un formulaire d'inscription complet `RegistrationForm`

Partie 1 - Sur papier (30 min) :

Concevoir le formulaire avec les champs suivants :

- **Nom complet** (input text) - obligatoire, min 3 caractères
- **Email** (input email) - obligatoire, format valide
- **Mot de passe** (input password) - obligatoire, min 8 caractères
- **Confirmation mot de passe** (input password) - doit correspondre
- **Date de naissance** (input date) - obligatoire
- **Genre** (radio buttons) - Homme / Femme / Autre
- **Pays** (select) - Liste de 5 pays au choix
- **Bio** (textarea) - max 200 caractères, compteur affiché
- **Photo de profil** (input file) - image uniquement, prévisualisation
- **Intérêts** (checkboxes) - Sport / Musique / Lecture / Voyage (multiple)
- **CGU** (checkbox) - obligatoire pour valider
- **Newsletter** (checkbox) - optionnel

Sur papier, dessiner :

1. Structure du state (objet `formData`)
2. Liste des validations à implémenter
3. Schéma du flux de données (`onChange`, re-render, user action, `setState`)

Partie 2 - Sur machine (30 min) :

Implémenter le formulaire avec :

- State initial avec tous les champs
- Handler `handleChange` unique pour tous les inputs (sauf file/checkbox)
- Handler `handleCheckboxChange` pour les checkboxes multiples
- Handler `handleFileChange` pour l'upload photo avec prévisualisation
- Validation à la soumission (fonction `validateForm`)
- Affichage des erreurs en rouge sous chaque champ
- Compteur caractères pour bio (ex : "45 / 200")
- Bouton "S'inscrire" désactivé si CGU non acceptées
- `Console.log` des données si formulaire valide

Critères de réussite :

- Tous les champs sont contrôlés (liés au state)
- Validation complète fonctionne
- Erreurs affichées clairement
- Upload image avec prévisualisation
- Compteur bio en temps réel
- Bouton submit désactivé si CGU non cochées
- Code propre et organisé

Bonus (+2 pts) :

- Validation en temps réel (`onChange`) plutôt qu'à la soumission
- Reset formulaire après soumission réussie
- Affichage récapitulatif données avant envoi